



CENTRO UNIVERSITÁRIO UNIRUY WYDEN
DISCIPLINA: ENGENHARIA DE SOFTWARE

**PLATAFORMA DE MONITORIAS ACADÊMICAS:
SISTEMA WEB PARA GESTÃO E INSCRIÇÃO EM MONITORIAS**

Aryane Santos Andrade
Rodrigo Santos Araújo

Salvador – BA
2026

**ARYANE SANTOS ANDRADE
RODRIGO SANTOS ARAÚJO**

**PLATAFORMA DE MONITORIAS ACADÊMICAS:
SISTEMA WEB PARA GESTÃO E INSCRIÇÃO EM MONITORIAS**

Trabalho acadêmico apresentado à disciplina
de Engenharia de Software ao Centro
Universitário UniRuy Wyden, como requisito
parcial de avaliação.

Professor: Heleno Cardoso

Salvador – BA
2026

RESUMO

O processo de gestão e divulgação de monitorias acadêmicas em instituições de ensino superior apresenta, frequentemente, características de descentralização e informalidade, com registros realizados por meio de planilhas eletrônicas e divulgações em grupos de mensagens, o que compromete a visibilidade das oportunidades, dificulta o controle de inscrições e aumenta a suscetibilidade a erros operacionais. Diante desse contexto, o presente trabalho tem por objetivo documentar e descrever o desenvolvimento de uma plataforma web denominada Plataforma de Monitorias Acadêmicas, concebida para centralizar e modernizar a gestão de monitorias no âmbito do Centro Universitário UniRuy Wyden. A solução foi desenvolvida seguindo os princípios da Engenharia de Software, compreendendo as etapas de levantamento de requisitos, modelagem de dados, especificação de regras de negócio, implementação, testes, implantação e publicação em ambiente de nuvem. A arquitetura adotada é baseada no framework Django, com linguagem Python, banco de dados relacional PostgreSQL, contêinerização via Docker e Docker Compose, servidor de aplicação Gunicorn e proxy reverso Nginx, com implantação em instância Amazon EC2 na plataforma AWS. A plataforma disponibiliza funcionalidades de cadastro e divulgação pública de monitorias, inscrição de alunos com controle automático de vagas, painel de gestão para monitores e dashboard com métricas de desempenho. Os resultados obtidos demonstram a viabilidade técnica e operacional da solução, que substitui o processo manual e descentralizado por um sistema organizado, acessível e escalável, contribuindo para a melhoria dos processos acadêmico-administrativos da instituição.

Palavras-chave: Engenharia de Software. Plataforma Web. Django. Gestão de Monitorias. Docker.

SUMÁRIO

1. PDTI – PLANO DIRETOR DE TECNOLOGIA DA INFORMAÇÃO	4
1.1 Levantamento de Custos	5
Custos de Infraestrutura e Hospedagem	5
Custos de Desenvolvimento	6
Custos Operacionais e Precificação por UST	6
1.2 Estudo de Viabilidade	6
Viabilidade Técnica	6
Viabilidade Econômica	7
Viabilidade Operacional	7
Riscos Identificados e Estratégias de Mitigação	7
2. PROJETO TÉCNICO	8
2.1 Consultoria do Negócio	9
Contexto Institucional	9
Problema Identificado	9
Objetivo do Sistema	10
Escopo Implementado e Limitações Atuais	10
Benefícios Esperados	10
3. O&M – OPERAÇÃO E MANUTENÇÃO	11
3.1 Processos, Rotinas e Procedimentos	12
Operação Diária	12
Rotinas de Manutenção	12
Procedimentos de Implantação	13
Procedimentos de Backup e Recuperação	13
Tratamento de Incidentes Comuns	13
4. PROJETO LÓGICO	14
4.1 Levantamento de Requisitos	15
Requisitos Funcionais	15
Requisitos Não Funcionais	15
4.2 Modelo Ambiental	16
Introdução	16
Objetivos e Funções do Sistema	16
Diagrama de Contexto	16
Lista de Eventos e Requisitos Associados	17

Regras de Negócio	18
4.3 Especificação de Programas	18
Modelos de Dados	19
Formulários e Validações	19
Views e Comportamentos	19
Rotas Principais do Sistema.....	20
4.4 Diagrama de Fluxo de Dados – Nível Zero.....	21
4.5 Diagrama de Fluxo de Dados por Evento.....	21
Evento: Visualização de Monitorias	22
Evento: Inscrição em Monitoria	22
Evento: Criação de Monitoria.....	23
Evento: Acesso ao Dashboard	23
4.6 Modelagem de Dados Conceitual – MER.....	24
5. PROJETO FÍSICO	25
5.1 Arquitetura da Aplicação.....	26
Django – Framework de Aplicação	26
PostgreSQL – Banco de Dados Relacional	26
Docker e Docker Compose – Contêinerização	26
Gunicorn – Servidor de Aplicação WSGI	26
Nginx – Proxy Reverso e Servidor de Arquivos Estáticos	27
Amazon Web Services (AWS) – Infraestrutura de Nuvem.....	27
5.2 Diagrama Entidade-Relacionamento (DER)	28
5.3 Dicionário de Dados	29
5.4 Modelo Comportamental.....	30
Fluxo de Cadastro e Publicação de Monitoria	30
Fluxo de Inscrição de Aluno	31
5.5 Plano de Testes	32
Testes Unitários	32
Testes de Integração.....	33
Testes de Aceitação	33
Testes de Validação e Limitações Atuais	33
5.6 Plano de Treinamento.....	34
Treinamento de Usuários Finais (Alunos).....	34
Treinamento de Monitores.....	34
Treinamento da Equipe de TI (Administradores).....	34
5.7 Plano de Implantação	34

5.8 Layout de Telas	35
Tela de Listagem de Monitorias (monitorias.html)	35
Tela de Inscrição (inscricao.html)	36
Tela de Cadastro de Monitor (registro.html)	36
Tela de Login (login.html)	37
Tela de Criação de Monitoria (criar_monitoria.html)	38
Tela de Edição de Monitoria (editar_monitoria.html)	38
Tela de Confirmação de Exclusão (excluir_monitoria.html)	39
Tela de Lista de Inscritos (inscritos_monitoria.html)	40
Tela do Painel do Monitor (painel_monitor.html)	40
Tela do Dashboard (dashboard.html)	41
5.9 Telas de Diálogo	42
5.10 Layout de Relatórios	42
5.11 Especificação de Programas – Módulos do Sistema	43
6. IMPLEMENTAÇÃO	43
6.1 Desenvolvimento das Aplicações	44
Organização do Código e Estrutura dos Módulos	44
Arquitetura MVT e Decisões Arquiteturais	44
6.2 Testes QA	44
Estratégia de Testes e Verificação	45
Qualidade do Software e Conformidade com Requisitos	45
7. IMPLANTAÇÃO	45
7.1 Instalação	46
7.2 Migração e Conversão de Dados	46
7.3 Treinamento	46
7.4 Validação	46
8. PRODUÇÃO	47
8.1 Homologação e Funcionamento do Sistema	48
CONCLUSÃO	48
REFERÊNCIAS	50

1. PDTI – PLANO DIRETOR DE TECNOLOGIA DA INFORMAÇÃO

O Plano Diretor de Tecnologia da Informação (PDTI) constitui o instrumento de orientação estratégica que define, organiza e prioriza as ações e investimentos em tecnologia de informação de uma organização. No contexto do presente projeto, o PDTI foi elaborado com o propósito de fundamentar as decisões técnicas e econômicas que nortearam o desenvolvimento da Plataforma de Monitorias Acadêmicas, assegurando alinhamento entre as necessidades institucionais e as soluções tecnológicas adotadas. As subseções a seguir detalham o levantamento de custos associados à operação da plataforma e o estudo de viabilidade que subsidiou a tomada de decisão para o seu desenvolvimento.

1.1 Levantamento de Custos

O levantamento de custos da plataforma abrange as dimensões de infraestrutura computacional em nuvem, hospedagem, desenvolvimento e operação continuada. A solução foi concebida para operar em ambiente de computação em nuvem da Amazon Web Services (AWS), utilizando instâncias de computação, volumes de armazenamento, banco de dados e serviços de rede. Os valores apresentados são estimativas baseadas na configuração mínima viável para suportar um ambiente acadêmico de pequeno a médio porte.

Custos de Infraestrutura e Hospedagem

A infraestrutura computacional adotada utiliza os seguintes serviços da plataforma AWS, com estimativas de custo mensais em dólares americanos (USD):

Item	Descrição	Estimativa Mensal (USD)
Computação	Instância EC2 t3.medium (2 vCPU, 4 GB RAM)	US\$ 40,00
Armazenamento	Volume EBS de 50 GB (gp3)	US\$ 5,00
Banco de Dados	PostgreSQL self-hosted no EC2 ou RDS t3.micro	US\$ 30,00 – 50,00
Backup	Armazenamento S3 para dumps do banco de dados	US\$ 3,00

Item	Descrição	Estimativa Mensal (USD)
Rede e DNS	Route53 e IP Elástico	US\$ 2,00
TOTAL ESTIMADO		US\$ 80,00 – 100,00

Custos de Desenvolvimento

O desenvolvimento da plataforma foi realizado exclusivamente com tecnologias de software livre e de código aberto, eliminando custos de licenciamento. As tecnologias utilizadas — Python, Django, PostgreSQL, Docker, Nginx e Gunicorn — possuem licenças permissivas que autorizam uso irrestrito em projetos acadêmicos e comerciais. O custo de desenvolvimento corresponde, portanto, ao investimento em horas de trabalho técnico da equipe, estimadas em aproximadamente 120 horas ao longo do ciclo de desenvolvimento.

Custos Operacionais e Precificação por UST

Para fins de estimativa acadêmica, adotou-se a métrica de Unidade de Serviço Técnico (UST) para precificação do esforço de desenvolvimento. Considerando uma UST equivalente a uma hora de trabalho técnico qualificado com valor de referência de R\$ 100,00/hora, o custo total de desenvolvimento pode ser estimado em aproximadamente R\$ 12.000,00 (doze mil reais) para o ciclo completo, compreendendo análise de requisitos, modelagem, implementação, testes e implantação. Os custos de manutenção mensal, incluindo monitoramento, aplicação de atualizações e suporte técnico, estimam-se em 20 UST mensais, equivalentes a R\$ 2.000,00/mês.

1.2 Estudo de Viabilidade

O estudo de viabilidade foi conduzido sob três perspectivas complementares: técnica, econômica e operacional. A análise integrada dessas dimensões forneceu o embasamento necessário para a decisão de desenvolvimento da solução proposta.

Viabilidade Técnica

Do ponto de vista técnico, a solução demonstra plena viabilidade, tendo sido desenvolvida com tecnologias maduras, amplamente documentadas e com comunidades ativas de suporte. O framework Django, na linguagem Python, oferece robustez para aplicações

web de médio e grande porte, com suporte nativo a padrões como MTV (Model-Template-View), ORM (Object-Relational Mapping) e sistema de autenticação. O banco de dados PostgreSQL é reconhecido pela comunidade técnica como uma das soluções de banco de dados relacional de maior confiabilidade e desempenho. A contêinerização por meio de Docker e Docker Compose assegura portabilidade entre ambientes de desenvolvimento e produção, reduzindo problemas de configuração. O servidor de aplicação Gunicorn e o proxy reverso Nginx constituem a combinação padrão de mercado para deploy de aplicações Django em produção. A implantação em instâncias Amazon EC2 da AWS garante escalabilidade, disponibilidade e flexibilidade de configuração.

Viabilidade Econômica

A viabilidade econômica da solução é sustentada pelo uso exclusivo de software livre, que elimina custos de licenciamento. O custo de infraestrutura estimado entre US\$ 80,00 e US\$ 100,00 mensais é compatível com orçamentos de tecnologia de informação de instituições acadêmicas de pequeno e médio porte. Em comparação com soluções proprietárias de gestão acadêmica, que frequentemente exigem contratos de licença de elevado valor, a plataforma proposta representa uma alternativa economicamente vantajosa, com total autonomia sobre o código-fonte e possibilidade de evolução sem dependência de fornecedores.

Viabilidade Operacional

A viabilidade operacional é garantida pela simplicidade do processo de instalação e manutenção da plataforma. O ambiente é completamente reproduzível mediante a execução do comando `docker compose up -d --build`, com configuração centralizada por meio de variáveis de ambiente documentadas nos arquivos `.env.example` e `.env.example-prod`. O script `entrypoint.sh` automatiza o processo de inicialização, incluindo a verificação de disponibilidade do banco de dados, execução de migrações de schema e coleta de arquivos estáticos. A manutenção técnica requer familiaridade com Docker, Django e PostgreSQL, competências comuns em equipes de tecnologia da informação acadêmicas.

Riscos Identificados e Estratégias de Mitigação

Os principais riscos identificados e as respectivas estratégias de mitigação adotadas são os seguintes:

- Risco de falha na inicialização do banco de dados: mitigado por healthchecks no Docker Compose e espera ativa implementada no `entrypoint.sh`, que aguarda a disponibilidade do serviço de banco de dados antes de iniciar a aplicação.
- Risco de inconsistência de dados de inscrição: mitigado por validações implementadas nos formulários Django e por constraint de unicidade definida no nível do banco de dados, impedindo inscrições duplicadas por e-mail em uma mesma monitoria.
- Risco de problemas de permissão em volumes de arquivos estáticos: mitigado por ajuste explícito de proprietário do diretório `staticfiles` no script `entrypoint.sh`.
- Risco de exposição de credenciais: mitigado pelo uso de variáveis de ambiente e pela não inclusão de arquivos `.env` no controle de versão, conforme configurado no `.gitignore` do repositório.

2. PROJETO TÉCNICO

2.1 Consultoria do Negócio

Contexto Institucional

O Centro Universitário UniRuy Wyden, instituição de ensino superior privada localizada na cidade de Salvador, estado da Bahia, conduz programas de monitoria acadêmica como parte integrante de sua política de apoio pedagógico. As monitorias constituem mecanismo de suporte didático em que estudantes com desempenho acadêmico destacado em determinadas disciplinas auxiliam colegas com maiores dificuldades de aprendizagem, sob supervisão docente. Tais programas são reconhecidos como instrumentos relevantes de formação complementar tanto para o monitor, que aprofunda seus conhecimentos ao ensiná-los, quanto para o aluno monitorado, que recebe atenção individualizada.

Problema Identificado

A análise do processo vigente de gestão de monitorias identificou um conjunto de deficiências estruturais que comprometem a eficiência e a transparência do programa. O processo era conduzido de forma integralmente descentralizada, com as seguintes características problemáticas:

- Divulgação das monitorias disponíveis realizada exclusivamente por meio de grupos de aplicativos de mensagens instantâneas, sem padronização de formato ou garantia de alcance a todos os alunos interessados.
- Controle das inscrições e das vagas realizado por meio de planilhas eletrônicas, sujeitas a erros de edição manual, duplicidade de registros e inconsistências de dados.
- Ausência de centralização das informações, tornando impossível a obtenção de uma visão consolidada das monitorias disponíveis, das vagas preenchidas e dos alunos inscritos.
- Dificuldade de gerenciamento por parte dos monitores, que dependiam de comunicação informal e processamento manual para controlar as inscrições recebidas.

- Baixa visibilidade das oportunidades de monitoria para o corpo discente, especialmente para alunos que não participavam dos grupos de comunicação em que as informações eram divulgadas.

Objetivo do Sistema

Diante do problema identificado, o sistema foi concebido com o objetivo de centralizar o cadastro, a divulgação e a gestão das monitorias acadêmicas em uma plataforma web acessível a todos os membros da comunidade acadêmica. A plataforma permite que monitores autenticados cadastrem e gerenciem suas monitorias, enquanto alunos podem consultar as oportunidades disponíveis e realizar inscrições de forma autônoma, com controle automático de disponibilidade de vagas.

Escopo Implementado e Limitações Atuais

O escopo funcional implementado na versão atual da plataforma compreende as seguintes funcionalidades: listagem pública de monitorias com busca por título e filtro por categoria; cadastro de monitorias por monitores autenticados; edição e exclusão de monitorias exclusivamente pelo monitor criador; inscrição de alunos em monitorias com validação de disponibilidade de vagas; painel de monitor com visualização de monitorias e lista de inscritos; e dashboard com métricas de desempenho e gráfico de inscrições por monitor.

Reconhecem-se como limitações do sistema em sua versão atual as seguintes características: ausência de geração de relatórios estruturados em formatos PDF ou CSV; ausência de sistema de notificação por e-mail; gerenciamento de categorias e benefícios dependente da interface administrativa do Django (Django Admin), sem interface CRUD dedicada; ausência de suporte a fila de espera ou cancelamento de inscrições; e inexistência de pipeline de integração e entrega contínua.

Benefícios Esperados

A implantação da plataforma projeta a obtenção dos seguintes benefícios para a instituição e seus membros: eliminação do processo manual de controle de vagas e inscrições; ampliação da visibilidade das monitorias disponíveis para todo o corpo discente; redução do risco de erros operacionais decorrentes do tratamento manual de dados; melhoria da experiência do aluno no processo de inscrição em monitorias; e

disponibilização de dados consolidados para a gestão acadêmica, por meio do dashboard de métricas.

3. O&M – OPERAÇÃO E MANUTENÇÃO

3.1 Processos, Rotinas e Procedimentos

Operação Diária

A rotina operacional diária da plataforma compreende um conjunto de verificações essenciais para assegurar a disponibilidade e o correto funcionamento do sistema em ambiente de produção. Os procedimentos a seguir devem ser executados pela equipe de operações:

- Verificação do status dos containers Docker (db, web e nginx) por meio do comando `docker compose ps`, confirmando que todos os serviços encontram-se no estado `running`.
- Análise dos arquivos de log gerados pelo Django, pelo Gunicorn e pelo Nginx, com especial atenção a erros de nível `ERROR` e `WARNING` que possam indicar falhas funcionais ou de desempenho.
- Teste de acessibilidade da rota pública `/monitorias/` por meio de requisição HTTP, verificando o código de resposta 200 e o conteúdo esperado da página.
- Confirmação de integridade dos registros de inscrições e do controle de vagas, verificando a consistência entre o número de vagas informado nas monitorias e o total de inscrições registradas no banco de dados.

Rotinas de Manutenção

A manutenção preventiva da plataforma deve seguir o cronograma de atividades descrito a seguir:

- Atualização de dependências: verificação e atualização das bibliotecas Python listadas no arquivo `requirements.txt` em ambiente de desenvolvimento, seguida de validação por meio da suíte de testes antes da promoção para produção.
- Backup periódico do banco de dados: execução de dumps do PostgreSQL em intervalos regulares (mínimo diário), com armazenamento em volume externo ou serviço S3 da AWS.
- Auditoria de permissões: verificação periódica das permissões do volume `staticfiles` para assegurar que os arquivos estáticos são acessíveis pelo Nginx.

- Revisão de variáveis de ambiente: conferência das definições de variáveis de ambiente nos arquivos .env e .env.example-prod para identificar configurações desatualizadas ou inadequadas para o ambiente de produção.

Procedimentos de Implantação

A implantação da plataforma em novos ambientes ou a atualização de versões existentes segue o procedimento padronizado descrito a seguir:

1. Clonar o repositório do sistema a partir do repositório GitHub do projeto.
2. Copiar o arquivo .env.example (desenvolvimento) ou .env.example-prod (produção) para .env e ajustar os valores das variáveis conforme o ambiente de destino.
3. Executar docker compose up -d --build (desenvolvimento) ou docker compose -f docker-compose.prod.yml up -d --build (produção) para inicializar os containers.
4. Verificar a execução automática do entryptoint.sh, que realiza a espera pela disponibilidade do banco de dados, executa as migrações de schema e coleta os arquivos estáticos.
5. Confirmar o funcionamento do sistema acessando a rota pública /monitorias/ e verificando as respostas esperadas.

Procedimentos de Backup e Recuperação

O procedimento de backup do banco de dados deve ser realizado por meio da execução do seguinte comando: docker compose exec db pg_dump -U <usuario> <nome_banco> > backup_AAAA-MM-DD.sql. O arquivo de dump gerado deve ser transferido para armazenamento externo seguro. A recuperação pode ser realizada por meio do comando: docker compose exec -T db psql -U <usuario> <nome_banco> < backup_AAAA-MM-DD.sql.

Tratamento de Incidentes Comuns

Os incidentes mais frequentemente identificados e seus respectivos procedimentos de resolução são:

- Erro no formulário de inscrição: verificar se a monitoria atingiu o limite de vagas (propriedade `monitoria_lotada`) e se o e-mail informado já se encontra cadastrado para a mesma monitoria (constraint de unicidade).
- Inacessibilidade do painel do monitor: confirmar autenticação do usuário na sessão ativa e verificar se o usuário é o proprietário (monitor) das monitorias exibidas.
- Container web não inicia: verificar a disponibilidade do serviço de banco de dados e a correção das variáveis de ambiente `DATABASE_URL` ou equivalentes definidas no arquivo `.env`.

4. PROJETO LÓGICO

4.1 Levantamento de Requisitos

O levantamento de requisitos constitui etapa fundamental do processo de Engenharia de Software, sendo responsável pela identificação, análise e documentação das necessidades dos usuários e das restrições do sistema. Os requisitos foram coletados mediante análise do processo vigente de gestão de monitorias na instituição e organizados nas categorias de requisitos funcionais e não funcionais.

Requisitos Funcionais

Os requisitos funcionais descrevem as funcionalidades que o sistema deve disponibilizar aos seus usuários:

Código	Descrição
RF01	O sistema deve exibir lista pública de monitorias com busca por título e filtro por categoria.
RF02	O sistema deve permitir a inscrição de aluno em monitoria por meio de formulário público.
RF03	O sistema deve permitir a criação de monitoria por usuário autenticado.
RF04	O sistema deve permitir a edição de monitoria exclusivamente pelo monitor criador.
RF05	O sistema deve permitir a exclusão de monitoria exclusivamente pelo monitor criador.
RF06	O sistema deve exibir a lista de inscritos por monitoria ao monitor autenticado proprietário.
RF07	O sistema deve permitir o registro de usuário e a autenticação por username ou e-mail.
RF08	O sistema deve exibir dashboard com métricas consolidadas e gráfico de inscrições por monitor.

Requisitos Não Funcionais

Os requisitos não funcionais descrevem as características de qualidade, desempenho e restrições operacionais do sistema:

Código	Descrição
RNF01	O sistema deve ser executável em contêiner Docker, garantindo portabilidade entre ambientes.
RNF02	O sistema deve utilizar PostgreSQL como sistema gerenciador de banco de dados relacional.
RNF03	O sistema deve permitir acesso público à listagem de monitorias sem exigência de autenticação.
RNF04	O sistema deve exigir autenticação de usuário para operações de criação, edição e exclusão de monitorias.
RNF05	O sistema deve garantir unicidade de inscrição por endereço de e-mail em cada monitoria.
RNF06	O sistema deve armazenar arquivos estáticos por meio do mecanismo collectstatic em volume dedicado.

4.2 Modelo Ambiental

Introdução

O modelo ambiental descreve o contexto em que o sistema está inserido, identificando os atores externos que interagem com ele, os fluxos de informação entre esses atores e o sistema, e os eventos que disparam os processos internos. A análise ambiental fornece uma visão sistêmica da plataforma, permitindo compreender suas fronteiras e suas interfaces com o mundo externo.

Objetivos e Funções do Sistema

O sistema tem como objetivos e funções centrais: centralizar a divulgação de monitorias acadêmicas disponíveis; controlar automaticamente a disponibilidade de vagas em cada monitoria; oferecer painel de gestão para monitores autenticados; garantir persistência segura e consistente dos dados em banco relacional; e facilitar a implantação e a manutenção por meio de contêineres Docker.

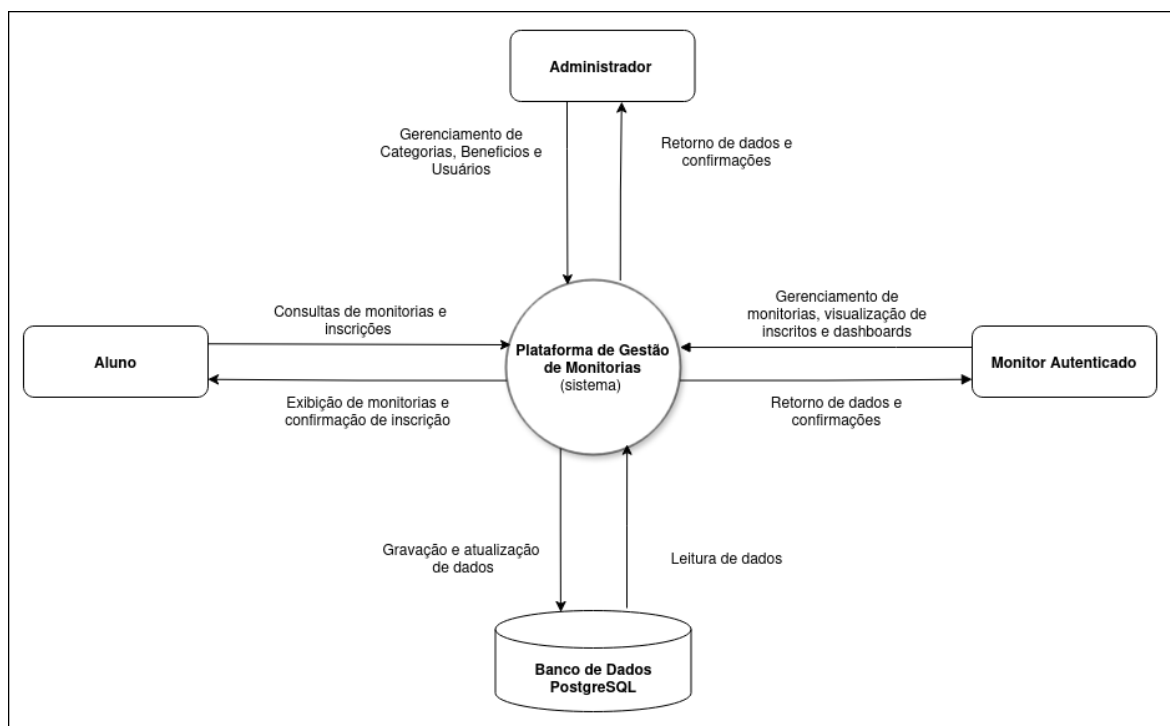
Diagrama de Contexto

O sistema interage com os seguintes atores externos:

- Visitante Público (Aluno): ator que acessa a plataforma sem autenticação para consultar monitorias disponíveis e realizar inscrições.

- Monitor Autenticado: ator que, após autenticação, gerencia suas monitorias (criar, editar, excluir) e visualiza a lista de alunos inscritos.
- Administrador do Sistema: ator com acesso à interface administrativa do Django (/admin/) para gerenciamento de categorias, benefícios e usuários.
- PostgreSQL: componente externo ao processo principal que armazena persistentemente todos os dados de domínio do sistema.

Figura 1 – Diagrama de Contexto



Fonte: Autores, 2026.

Lista de Eventos e Requisitos Associados

Evento	Descrição	Requisito Associado
Acesso à listagem de monitorias	Visitante acessa a rota /monitorias/	RF01
Submissão de inscrição	Visitante submete formulário de inscrição em monitoria	RF02
Registro de usuário	Novo monitor realiza cadastro na plataforma	RF07

Evento	Descrição	Requisito Associado
Autenticação de usuário	Monitor realiza login por username ou e-mail	RF07
Criação de monitoria	Monitor autenticado cadastra nova monitoria	RF03
Edição de monitoria	Monitor autenticado edita dados de sua monitoria	RF04
Exclusão de monitoria	Monitor autenticado remove sua monitoria	RF05
Visualização de inscritos	Monitor acessa lista de alunos inscritos em sua monitoria	RF06
Acesso ao dashboard	Monitor autenticado acessa painel de métricas	RF08

Regras de Negócio

As regras de negócio definem as políticas e restrições que governam o comportamento do sistema:

- RN01: A inscrição em uma monitoria somente é permitida se a monitoria não estiver lotada, ou seja, se o número de inscrições registradas for inferior ao número total de vagas definido.
- RN02: Um endereço de e-mail não pode ser utilizado para mais de uma inscrição em uma mesma monitoria, sendo esta restrição implementada como constraint de unicidade no banco de dados.
- RN03: O monitor não pode realizar inscrição em sua própria monitoria.
- RN04: Somente o usuário associado ao campo monitor de uma Monitoria pode alterar ou remover o respectivo registro.
- RN05: A ocupação de vagas é calculada dinamicamente pelo número de registros de Inscrição vinculados à Monitoria, por meio das propriedades vagas_preenchidas e vagas_restantes.

4.3 Especificação de Programas

A especificação de programas detalha os modelos de dados, formulários, views e rotas que compõem o sistema, descrevendo o comportamento esperado de cada componente.

Modelos de Dados

O sistema é composto pelos seguintes modelos Django, que representam as entidades do domínio:

- Categoria: entidade de classificação com campos id (chave primária) e nome (nome da categoria).
- Beneficio: entidade de classificação com campos id (chave primária) e nome (tipo de beneficio oferecido).
- Monitoria: entidade central com campos id, titulo, descricao, data, categoria (FK), beneficio (FK), vagas e monitor (FK para User). Disponibiliza as propriedades computadas vagas_preenchidas, vagas_restantes e monitoria_lotada.
- Inscricao: entidade de inscrição com campos id, monitoria (FK), nome, email e telefone. Aplica constraint de unicidade sobre o par (monitoria, email).

Formulários e Validações

Os formulários implementados e suas respectivas validações são:

- MonitoriaModelForm: formulário de cadastro e edição de monitoria, sem exposição do campo monitor (preenchido automaticamente com o usuário autenticado).
- InscricaoModelForm: formulário de inscrição pública, com validação que impede duplicidade de e-mail por monitoria e verifica disponibilidade de vagas.
- RegisterForm: formulário de cadastro de novo usuário, com validação de unicidade do endereço de e-mail.
- LoginForm: formulário de autenticação com suporte a login por e-mail ou username, mediante backend de autenticação personalizado EmailOrUsernameBackend.

Views e Comportamentos

As views implementadas e seus comportamentos funcionais são:

- `MonitoriaListView`: exibe a listagem pública de monitorias, aplicando filtros de busca por título e por categoria conforme parâmetros da querystring.
- `MonitoriaInscricaoView`: apresenta o formulário de inscrição e processa a submissão, validando disponibilidade de vagas e unicidade de e-mail.
- `NovaMonitoriaCreateView`: permite a criação de nova monitoria pelo monitor autenticado, associando automaticamente o usuário ao campo monitor.
- `MonitoriaUpdateView` e `MonitoriaDeleteView`: restringem as operações de edição e exclusão ao monitor proprietário, por meio de queryset filtrado.
- `MonitoriaInscritoListView`: exibe a lista de alunos inscritos em uma monitoria, acessível apenas ao monitor proprietário autenticado.
- `painel_view`: apresenta o painel consolidado do monitor com suas monitorias cadastradas.
- `dashboard_view`: realiza agregação de dados de inscrições por monitoria e gera os dados para o gráfico do dashboard.

Rotas Principais do Sistema

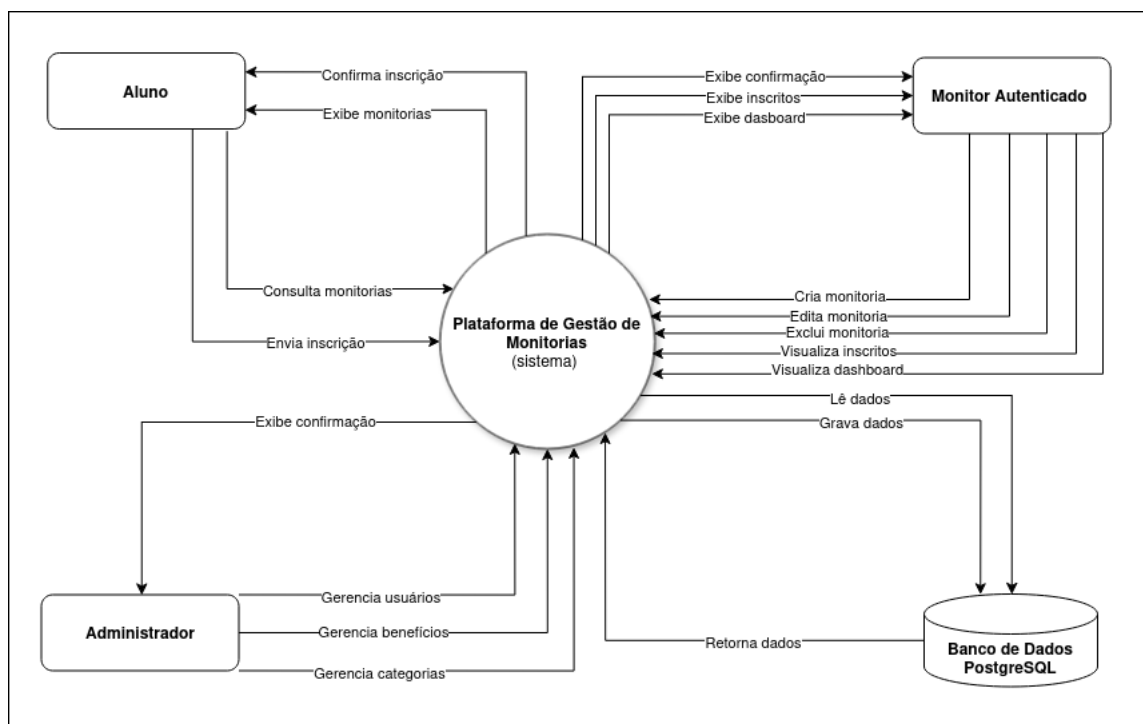
URL	View	Nível de Acesso
/monitorias/	<code>MonitoriaListView</code>	Público
/monitorias/<pk>/inscricao/	<code>MonitoriaInscricaoView</code>	Público
/criar_monitoria/	<code>NovaMonitoriaCreateView</code>	Autenticado
/painel/<pk>/editar_monitoria	<code>MonitoriaUpdateView</code>	Autenticado (proprietário)
/painel/<pk>/excluir_monitoria	<code>MonitoriaDeleteView</code>	Autenticado (proprietário)
/registro/	<code>register_view</code>	Público
/login/	<code>login_view</code>	Público
/logout/	<code>logout_view</code>	Autenticado
/painel/	<code>painel_view</code>	Autenticado
/dashboard/	<code>dashboard_view</code>	Autenticado

4.4 Diagrama de Fluxo de Dados – Nível Zero

O Diagrama de Fluxo de Dados (DFD) de nível zero representa o sistema como um único processo central, denominado Plataforma de Monitorias Acadêmicas, destacando os atores externos e os fluxos de dados entre eles e o sistema. Neste nível de abstração, o sistema é tratado como uma caixa-preta cujas entradas e saídas são identificadas.

Os fluxos de entrada identificados são: consulta de monitorias e dados de inscrição provenientes do Visitante Público; dados de monitoria (criação, edição, exclusão) e consultas ao painel e dashboard provenientes do Monitor Autenticado; e operações administrativas provenientes do Administrador do Sistema. Os fluxos de saída são: listas de monitorias e confirmações de inscrição para o Visitante; visualizações de painel, listas de inscritos e métricas do dashboard para o Monitor; e confirmações de operações administrativas para o Administrador. O sistema interage com o repositório de dados PostgreSQL para operações de leitura e gravação, retornando os dados processados aos respectivos atores.

Figura 2 – Diagrama de Fluxo de Dados (DFD) de nível zero



Fonte: Autores, 2026.

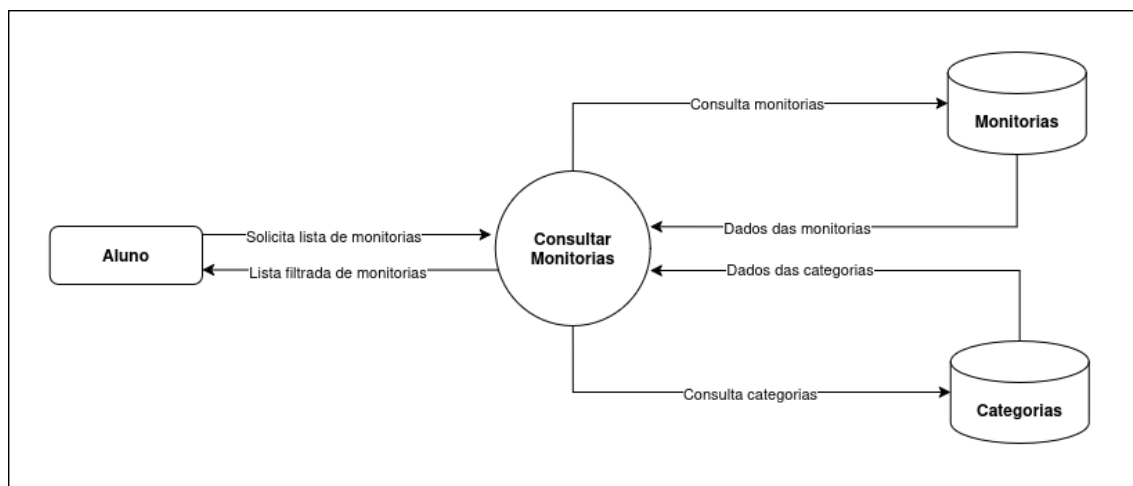
4.5 Diagrama de Fluxo de Dados por Evento

O DFD por evento detalha os fluxos de dados para cada evento identificado no modelo ambiental. Os principais fluxos são descritos a seguir.

Evento: Visualização de Monitorias

Entrada: requisição HTTP GET para a rota /monitorias/. Processamento: o método `get_queryset()` da `MonitoriaListView` aplica os filtros de busca por título e por categoria conforme os parâmetros presentes na `querystring` da requisição. Saída: página HTML renderizada com a lista de monitorias filtradas, incluindo informações de vagas disponíveis para cada monitoria.

Figura 3 – DFD por Evento – Visualização de Monitorias

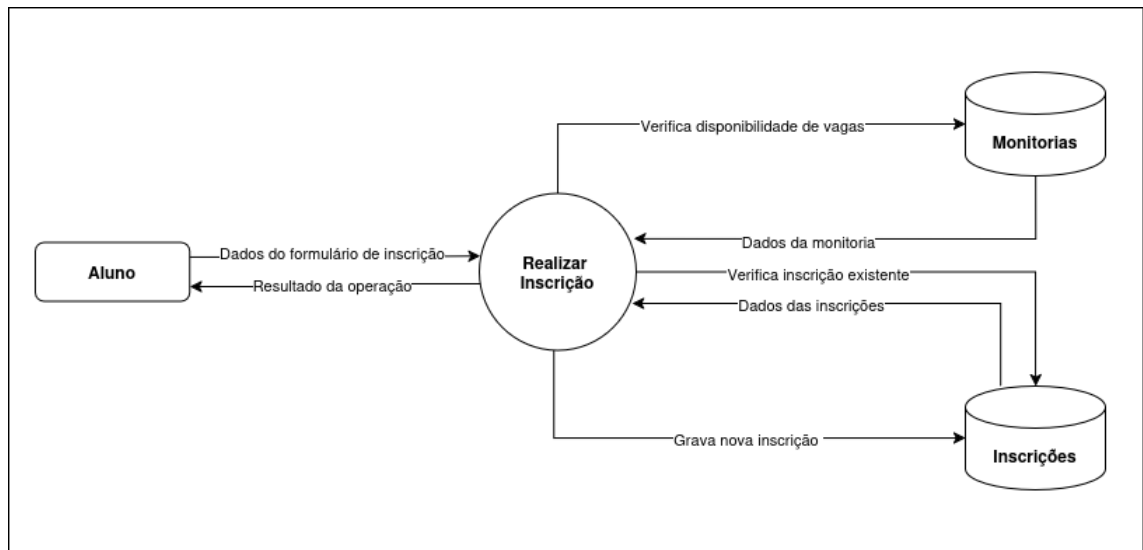


Fonte: Autores, 2026.

Evento: Inscrição em Monitoria

Entrada: requisição HTTP POST para a rota /monitorias/<pk>/inscricao/ com os dados do formulário de inscrição. Processamento: o método `post()` da `MonitoriaInscricaoView` instancia e valida o `InscricaoModelForm`, que executa as validações de disponibilidade de vagas (propriedade `monitoria_lotada`) e de unicidade de e-mail por monitoria (método `clean_email()`). Saída: inscrição gravada no banco de dados e mensagem de confirmação ao usuário, ou mensagem de erro com indicação da falha de validação.

Figura 4 – DFD por Evento – Inscrição em Monitoria

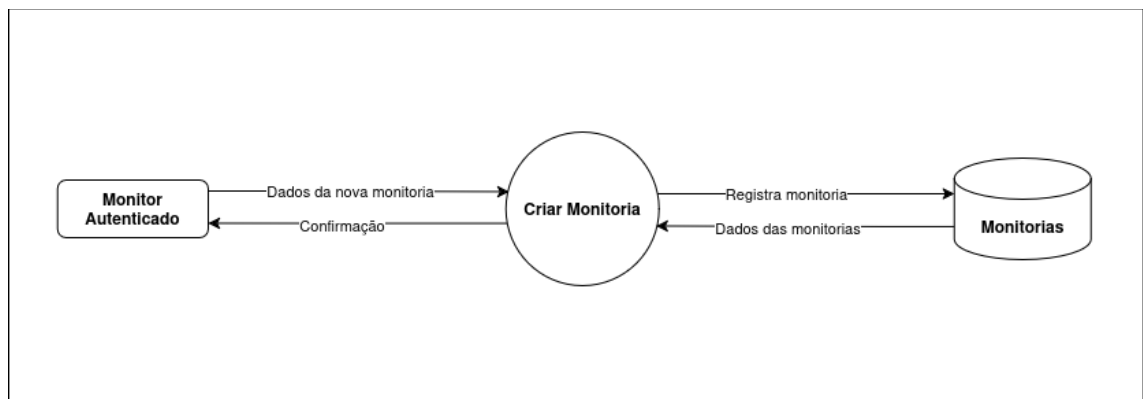


Fonte: Autores, 2026.

Evento: Criação de Monitoria

Entrada: requisição HTTP POST para a rota /criar_monitoria/ com os dados do formulário de monitoria. Processamento: o método `form_valid()` da `NovaMonitoriaCreateView` associa o usuário autenticado ao campo `monitor` do objeto `Monitoria` antes de persistir o registro. Saída: redirecionamento ao painel do monitor com a nova monitoria exibida.

Figura 5 – DFD por Evento – Criação de Monitoria



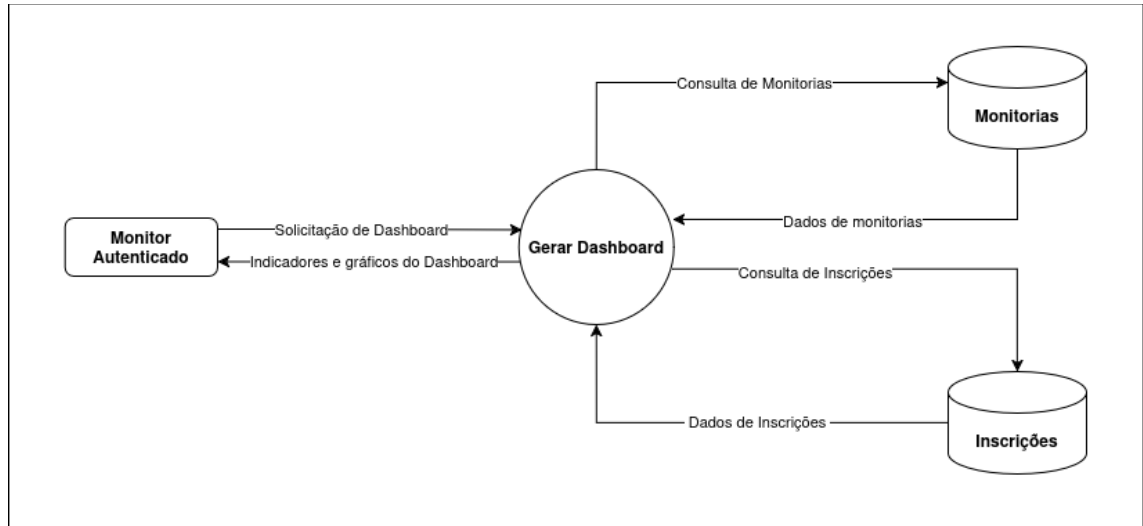
Fonte: Autores, 2026.

Evento: Acesso ao Dashboard

Entrada: requisição HTTP GET para a rota /dashboard/. Processamento: a `dashboard_view()` realiza agregação dos dados de inscrições por monitoria utilizando anotações do ORM do Django. Saída: página do dashboard com métricas consolidadas

(total de monitorias, total de inscrições, taxa de ocupação) e dados formatados para renderização do gráfico de barras.

Figura 6 – DFD por Evento – Acesso ao Dashboard



Fonte: Autores, 2026.

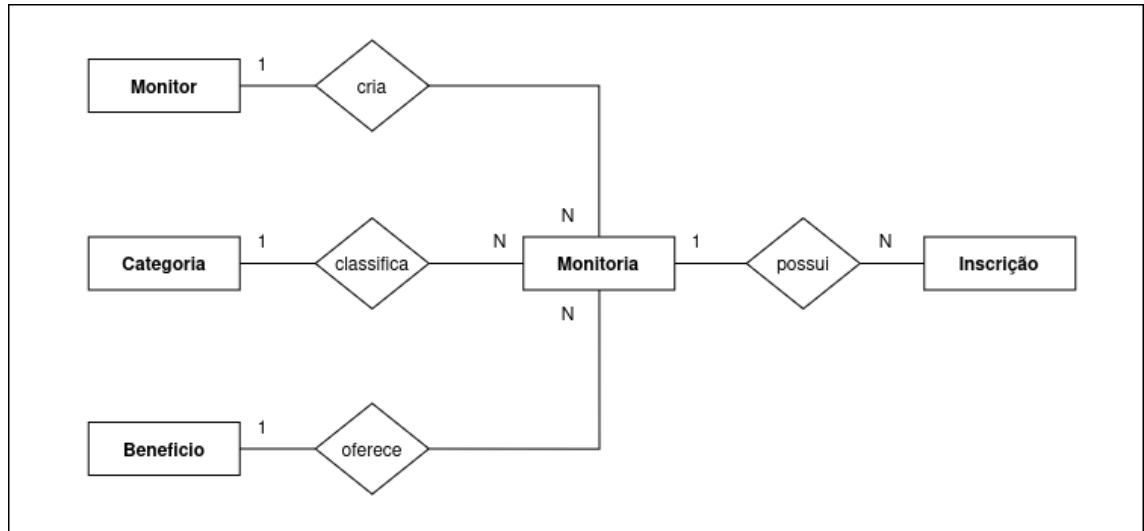
4.6 Modelagem de Dados Conceitual – MER

O Modelo Entidade-Relacionamento (MER) descreve as entidades do domínio e seus relacionamentos no nível conceitual. As entidades identificadas e seus relacionamentos são:

- **Usuário (User):** entidade fornecida pelo framework Django representando os monitores cadastrados na plataforma. Relaciona-se com Monitoria na cardinalidade 1:N (um usuário pode ser monitor de várias monitorias).
- **Monitoria:** entidade central do domínio, representando uma oferta de monitoria acadêmica. Relaciona-se com Categoria (N:1), Benefício (N:1), User (N:1) e Inscricao (1:N).
- **Inscricao:** entidade que registra a inscrição de um aluno em uma monitoria. Relaciona-se com Monitoria (N:1), com restrição de unicidade sobre o par (monitoria, email).
- **Categoria:** entidade de classificação das monitorias por área de conhecimento. Relaciona-se com Monitoria (1:N).

- **Benefício:** entidade que registra o tipo de benefício oferecido pela monitoria. Relaciona-se com Monitoria (1:N).

Figura 7 – Modelo Entidade-Relacionamento (MER)



Fonte: Autores, 2026.

5. PROJETO FÍSICO

5.1 Arquitetura da Aplicação

A arquitetura física da Plataforma de Monitorias Acadêmicas é constituída por um conjunto de componentes de software organizados em contêineres Docker, orquestrados pelo Docker Compose. Cada componente desempenha um papel específico e bem definido na cadeia de processamento das requisições.

Django – Framework de Aplicação

O Django, na versão 6, constitui o núcleo da aplicação, sendo responsável pelo processamento das requisições HTTP, pela lógica de negócio, pela interação com o banco de dados por meio de seu ORM e pela renderização das respostas HTML. O padrão arquitetural adotado é o MTV (Model-Template-View), variação do padrão MVC (Model-View-Controller) adaptada pelo framework Django, em que os Models definem a estrutura dos dados, os Templates definem a apresentação e as Views implementam a lógica de controle.

PostgreSQL – Banco de Dados Relacional

O PostgreSQL é utilizado como sistema gerenciador de banco de dados relacional, sendo responsável pela persistência de todos os dados do domínio da aplicação. Em ambiente de desenvolvimento, o banco de dados é executado em um container Docker dedicado. Em ambiente de produção, a configuração prevê a utilização de uma instância externa ao servidor de aplicação, seja um serviço gerenciado como o Amazon RDS ou uma instância EC2 dedicada, permitindo maior flexibilidade na gestão de recursos.

Docker e Docker Compose – Contêinerização

A contêinerização por meio do Docker e do Docker Compose assegura a portabilidade e a reprodutibilidade do ambiente de execução da aplicação entre diferentes estágios do ciclo de desenvolvimento: desenvolvimento local, homologação e produção. O arquivo docker-compose.yml define o ambiente de desenvolvimento, enquanto o docker-compose.prod.yml define o ambiente de produção, com configurações específicas de segurança e desempenho.

Gunicorn – Servidor de Aplicação WSGI

O Gunicorn (Green Unicorn) é o servidor de aplicação WSGI (Web Server Gateway Interface) utilizado para executar a aplicação Django em ambiente de produção. Por tratar-se de um servidor de aplicação multi-worker, o Gunicorn é capaz de processar múltiplas requisições concorrentes, diferentemente do servidor de desenvolvimento embutido do Django, que é de uso exclusivamente local. O Gunicorn é executado no container web e recebe requisições do Nginx por meio do protocolo HTTP na porta 8000.

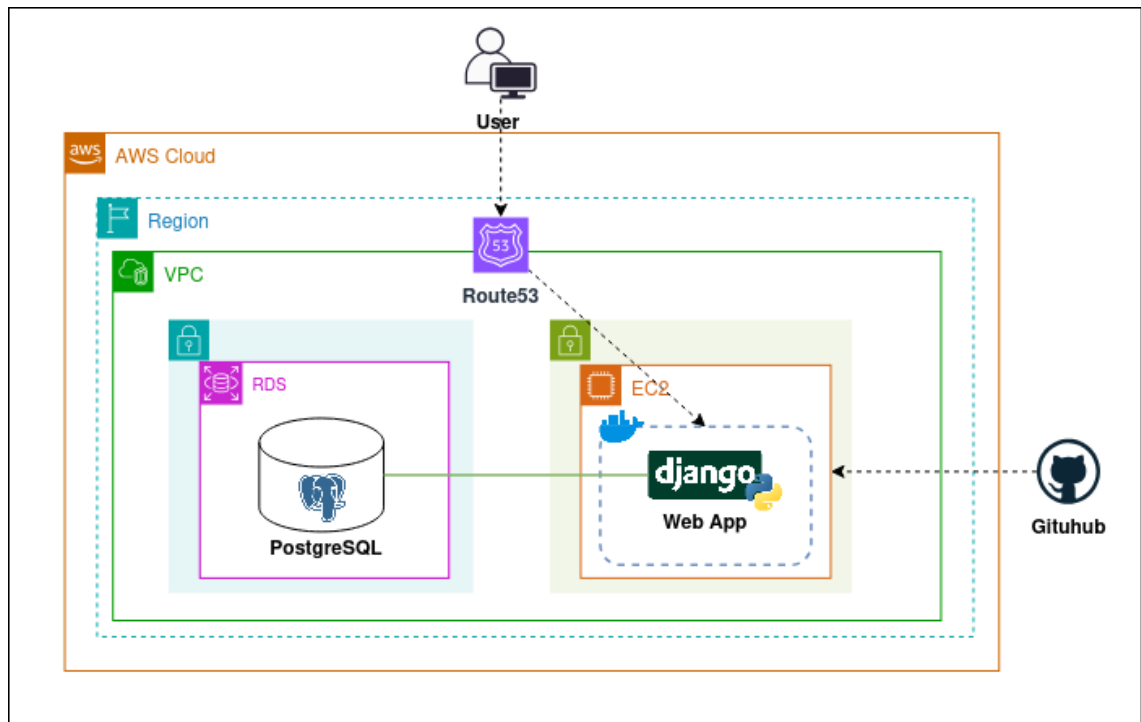
Nginx – Proxy Reverso e Servidor de Arquivos Estáticos

O Nginx desempenha duas funções complementares na arquitetura: servidor de proxy reverso e servidor de arquivos estáticos. Como proxy reverso, o Nginx recebe as requisições HTTP/HTTPS dos clientes na porta 443 (com TLS) ou 80 e as encaminha ao Gunicorn na porta 8000, adicionando cabeçalhos de identificação e gerenciando conexões de forma eficiente. Como servidor de arquivos estáticos, o Nginx serve diretamente os arquivos CSS, JavaScript e imagens coletados pelo collectstatic do Django, sem envolver o Gunicorn nesse processamento, o que resulta em melhor desempenho.

Amazon Web Services (AWS) – Infraestrutura de Nuvem

A implantação em produção utiliza a plataforma Amazon Web Services (AWS). A aplicação é executada em uma instância Amazon EC2 responsável pela hospedagem dos contêineres Docker que executam os serviços Django, Gunicorn e Nginx. A persistência dos dados é realizada por meio de uma instância Amazon RDS for PostgreSQL, permitindo a separação entre as camadas de aplicação e banco de dados. O acesso público ao sistema é disponibilizado por meio de um Elastic IP associado à instância EC2, enquanto o serviço Amazon Route 53 é utilizado para o gerenciamento do domínio e resolução DNS. Para garantir a segurança das comunicações, são utilizados certificados TLS emitidos pelo Let's Encrypt e gerenciados pelo Certbot, permitindo o acesso seguro à aplicação por meio do protocolo HTTPS.

Figura 8 – Diagrama de Arquitetura de Implantação em Nuvem na AWS



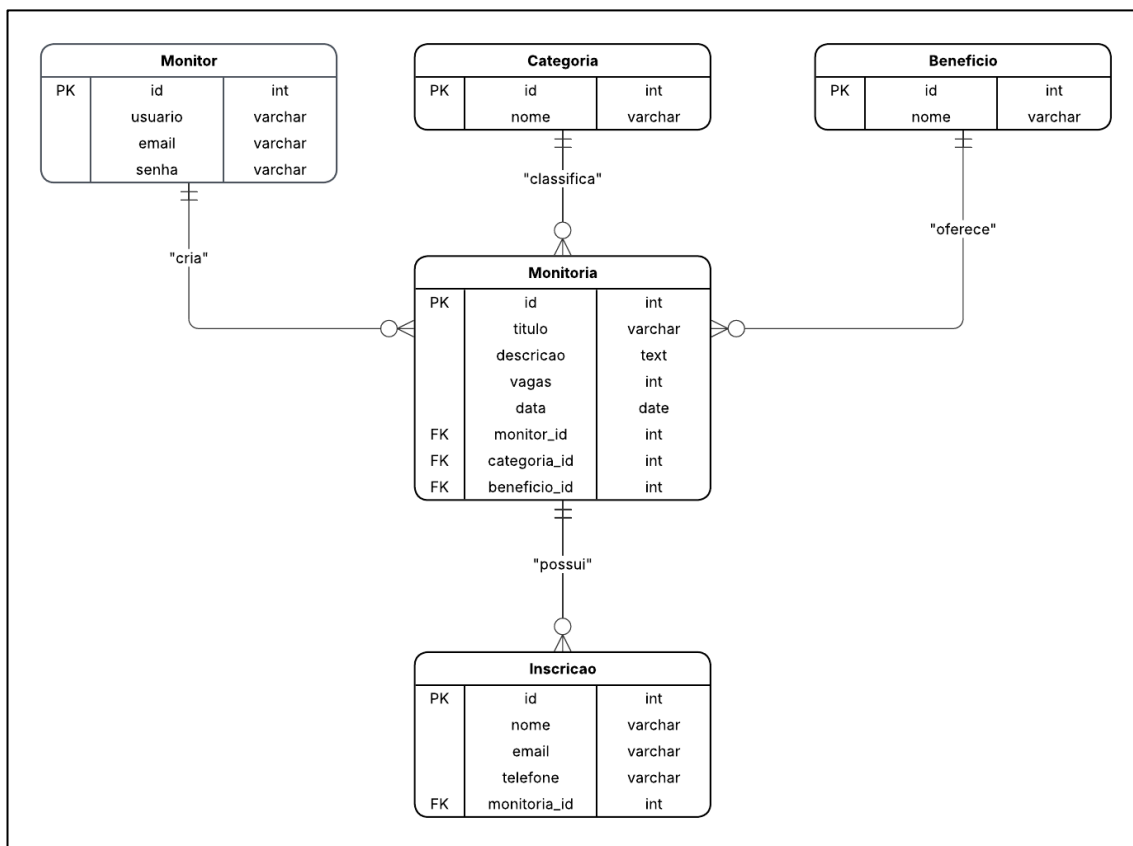
Fonte: Autores, 2026.

5.2 Diagrama Entidade-Relacionamento (DER)

O Diagrama Entidade-Relacionamento no nível físico descreve as tabelas, seus atributos e os relacionamentos implementados no banco de dados PostgreSQL. O modelo físico é composto pelas seguintes relações:

- Tabela Categoria: colunas id (PK, inteiro autoincremental) e nome (varchar, not null). Referenciada pela tabela Monitoria.
- Tabela Benefício: colunas id (PK, inteiro autoincremental) e nome (varchar, not null). Referenciada pela tabela Monitoria.
- Tabela Monitoria: colunas id (PK), titulo (varchar), descricao (text), data (date, nullable), categoria_id (FK → Categoria), beneficio_id (FK → Benefício), vagas (integer), monitor_id (FK → Monitor).
- Tabela Inscricao: colunas id (PK), monitoria_id (FK → Monitoria), nome (varchar), email (varchar), telefone (varchar). Constraint UNIQUE sobre o par (monitoria_id, email).
- Tabela Monitor: tabela nativa do Django, armazena os dados dos usuários (monitores) autenticados na plataforma.

Figura 9 – Diagrama Entidade-Relacionamento (DER)



Fonte: Autores, 2026.

5.3 Dicionário de Dados

O dicionário de dados descreve detalhadamente cada campo das entidades do sistema:

Entidade	Campo	Tipo	Restrições	Descrição
Categoria	id	Integer	PK, Auto	Identificador único da categoria
Categoria	nome	Varchar	Not Null	Nome descritivo da categoria
Beneficio	id	Integer	PK, Auto	Identificador único do beneficio
Beneficio	nome	Varchar	Not Null	Nome descritivo do beneficio
Monitoria	id	Integer	PK, Auto	Identificador único da monitoria

Entidade	Campo	Tipo	Restrições	Descrição
Monitoria	titulo	Varchar	Not Null	Título da monitoria
Monitoria	descricao	Text	Not Null	Descrição detalhada da monitoria
Monitoria	data	Date	Nullable	Data de realização da monitoria
Monitoria	categoria_id	Integer	FK, Not Null	Referência à Categoria
Monitoria	beneficio_id	Integer	FK, Not Null	Referência ao Benefício
Monitoria	vagas	Integer	Not Null	Número total de vagas da monitoria
Monitoria	monitor_id	Integer	FK, Not Null	Referência ao usuário Monitor
Inscricao	id	Integer	PK, Auto	Identificador único da inscrição
Inscricao	monitoria_id	Integer	FK, Not Null	Referência à Monitoria
Inscricao	nome	Varchar	Not Null	Nome completo do inscrito
Inscricao	email	Varchar	Not Null	E-mail do inscrito
Inscricao	telefone	Varchar	Nullable	Telefone de contato do inscrito

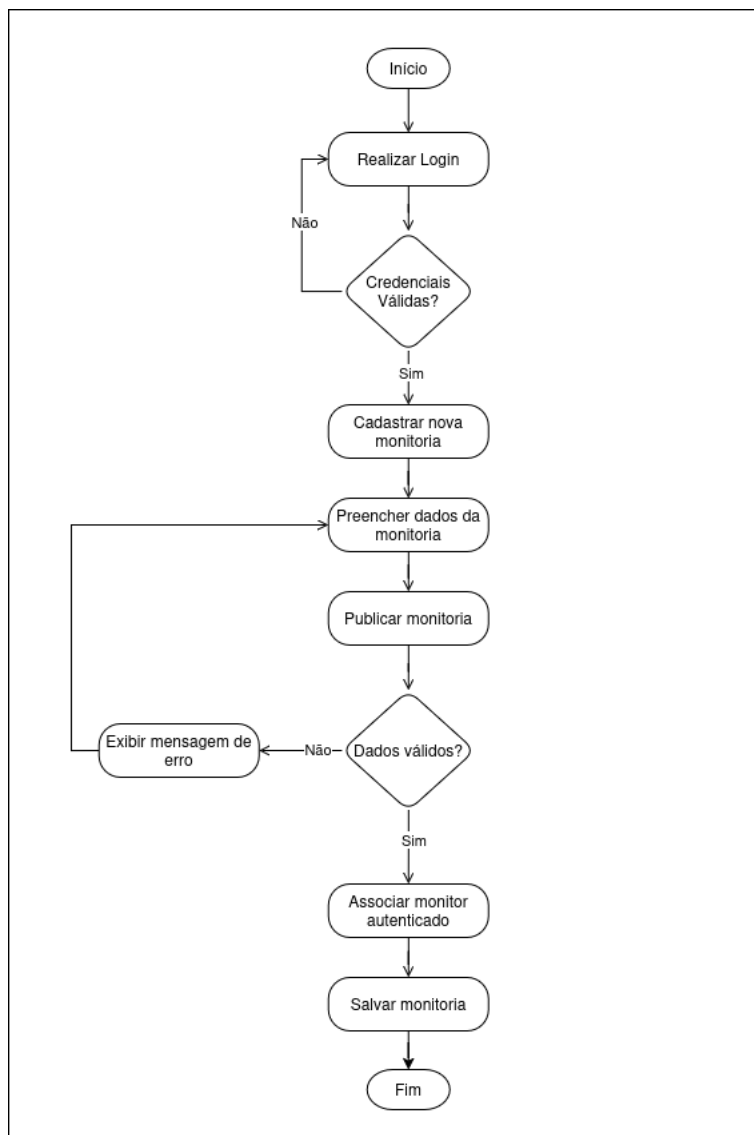
5.4 Modelo Comportamental

O modelo comportamental descreve os fluxos principais de interação do usuário com o sistema, representando as sequências de ações e respostas em cada caso de uso relevante.

Fluxo de Cadastro e Publicação de Monitoria

O monitor acessa a tela de login, insere suas credenciais (username ou e-mail e senha) e é autenticado. Após autenticação, acessa a rota `/criar_monitoria/`, preenche o formulário com título, descrição, data, categoria, benefício e número de vagas e submete o formulário. O sistema valida os dados, associa o usuário autenticado ao campo monitor e persiste o registro. O monitor é redirecionado ao painel, onde a nova monitoria é exibida.

Figura 10 – Fluxo de Cadastro e Publicação de Monitoria

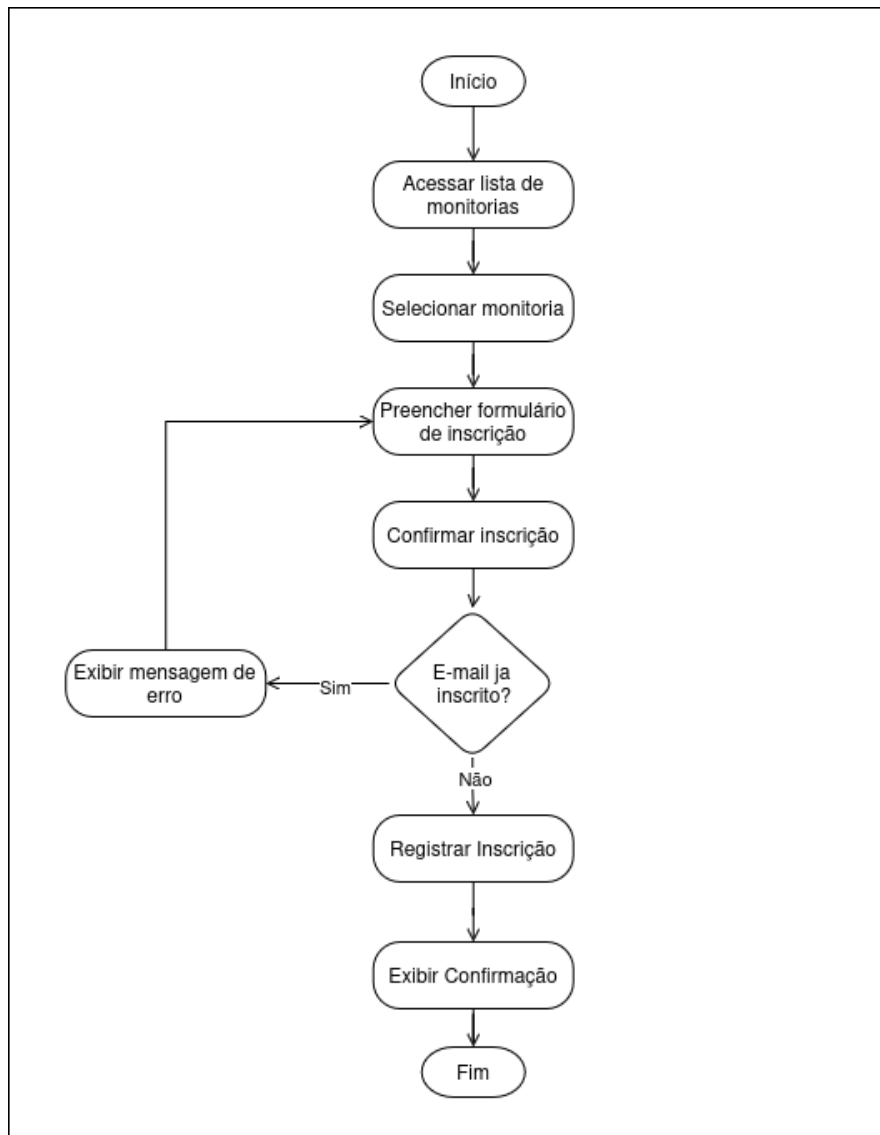


Fonte: Autores, 2026.

Fluxo de Inscrição de Aluno

O aluno acessa a rota /monitorias/ sem necessidade de autenticação, visualiza a lista de monitorias disponíveis e seleciona aquela de seu interesse. Na tela de inscrição, preenche nome, e-mail e telefone e submete o formulário. O sistema verifica se a monitoria possui vagas disponíveis e se o e-mail não está previamente cadastrado para a mesma monitoria. Em caso de aprovação, a inscrição é registrada e uma mensagem de confirmação é exibida. Em caso de rejeição, a mensagem de erro correspondente é apresentada ao aluno.

Figura 11 – Fluxo de Inscrição de Aluno



Fonte: Autores, 2026.

5.5 Plano de Testes

O plano de testes define a estratégia de verificação e validação do sistema, organizada nas categorias de testes unitários, testes de integração, testes de aceitação e testes de validação.

Testes Unitários

Os testes unitários verificam o comportamento isolado de componentes individuais do sistema. Os testes implementados na suíte de testes do Django cobrem:

- Validação das propriedades vagas_preenchidas, vagas_restantes e monitoria_lotada do modelo Monitoria.
- Restrição de unicidade de inscrição por e-mail em uma mesma monitoria, por meio do InscricaoModelForm.
- Comportamento da MonitoriaListView na listagem pública, verificando filtros de busca e categoria.
- Requisito de autenticação para acesso à NovaMonitoriaCreateView.
- Fluxos de registro e login de usuário, incluindo o backend customizado de autenticação.

Testes de Integração

Os testes de integração verificam a interação entre os componentes do sistema. No contexto da plataforma, os testes de integração prioritários incluem: a cadeia de requisição HTTP da view ao banco de dados, verificando a persistência correta dos dados; a integração entre o formulário de inscrição e as validações do modelo; e a integração entre a view do dashboard e as anotações do ORM do Django.

Testes de Aceitação

Os testes de aceitação validam o sistema sob a perspectiva do usuário final, verificando se as funcionalidades atendem aos requisitos funcionais especificados. Os critérios de aceitação definidos para cada requisito funcional são:

- RF01: A página /monitorias/ deve exibir corretamente a lista de monitorias com busca funcional por título e filtro por categoria.
- RF02: O formulário de inscrição deve registrar corretamente a inscrição e bloquear inscrições em monitorias lotadas ou com e-mail duplicado.
- RF03, RF04, RF05: As operações de CRUD de monitorias devem ser acessíveis apenas a monitores autenticados e restritas ao proprietário da monitoria.
- RF08: O dashboard deve exibir métricas consistentes com os dados registrados no banco de dados.

Testes de Validação e Limitações Atuais

A validação do sistema é realizada por meio da execução da suíte de testes Django com o comando `docker compose exec web python manage.py test`, complementada por testes manuais dos fluxos críticos. O plano atual não abrange testes de interface do painel do monitor, testes de integração de deploy, testes de desempenho ou testes de segurança (penetration testing). Esses aspectos constituem trabalhos futuros para a evolução do plano de qualidade da plataforma.

5.6 Plano de Treinamento

O plano de treinamento define os procedimentos e conteúdos necessários para habilitar os diferentes perfis de usuário da plataforma para sua correta utilização.

Treinamento de Usuários Finais (Alunos)

O treinamento de alunos é mínimo, dada a natureza pública e intuitiva da interface de inscrição. As orientações básicas compreendem: acesso à URL pública da plataforma; navegação pela lista de monitorias disponíveis; uso dos filtros de busca por título e categoria; preenchimento e submissão do formulário de inscrição.

Treinamento de Monitores

O treinamento de monitores deve abranger: processo de registro e autenticação na plataforma; cadastro de nova monitoria com todos os campos obrigatórios; edição e exclusão de monitorias existentes; visualização da lista de alunos inscritos em cada monitoria; e utilização do dashboard para acompanhamento das métricas de desempenho.

Treinamento da Equipe de TI (Administradores)

O treinamento da equipe técnica deve cobrir: processo de instalação e configuração do ambiente Docker; configuração das variáveis de ambiente para produção; execução de migrações de banco de dados; procedimentos de backup e recuperação; monitoramento de logs dos containers; e utilização da interface administrativa do Django para gerenciamento de categorias, benefícios e usuários.

5.7 Plano de Implantação

O plano de implantação descreve o processo completo de publicação da plataforma em ambiente de produção, conforme efetivamente realizado no projeto.

6. Provisionamento da instância EC2 na AWS com sistema operacional Ubuntu Server LTS.
7. Instalação do Docker Engine e do Docker Compose na instância EC2.
8. Clonagem do repositório do projeto a partir do GitHub na instância EC2.
9. Configuração do arquivo `.env` a partir do `.env.example-prod`, com definição de `SECRET_KEY`, credenciais do banco de dados, domínio e parâmetros de segurança.
10. Obtenção de certificado TLS via Certbot (Let's Encrypt) no modo `--webroot`, com armazenamento em `/etc/letsencrypt`.
11. Configuração do Nginx para servir o domínio `monitorias.andops.com.br` com redirecionamento HTTP para HTTPS e proxy reverso para o Gunicorn.
12. Inicialização dos containers com `docker compose -f docker-compose.prod.yml up -d --build`.
13. Verificação da execução do `entrypoint.sh`, incluindo migrações e `collectstatic`.
14. Validação do acesso público à plataforma pela URL de produção.

5.8 Layout de Telas

A interface da Plataforma de Monitorias Acadêmicas foi desenvolvida com foco na usabilidade, proporcionando uma navegação simples e intuitiva para monitores e alunos. As principais telas do sistema são descritas a seguir.

Tela de Listagem de Monitorias (monitorias.html)

Corresponde à tela inicial do sistema, responsável por apresentar as monitorias disponíveis para inscrição. As informações são exibidas em formato de cartões (*cards*), contendo título, categoria, benefício associado, quantidade de vagas disponíveis e opção para realização da inscrição. A interface disponibiliza ainda mecanismos de pesquisa por título e filtragem por categoria.

Figura 12 – Tela de Listagem de Monitorias



Fonte: Autores, 2026.

Tela de Inscrição (inscricao.html)

Apresenta os detalhes da monitoria selecionada e o formulário de inscrição do aluno. O formulário é composto pelos campos nome, e-mail e telefone, necessários para efetivação da inscrição.

Figura 13 – Tela de Inscrição em Monitoria



Fonte: Autores, 2026.

Tela de Cadastro de Monitor (registro.html)

Permite o registro de novos monitores na plataforma. A interface disponibiliza um formulário para cadastro das informações necessárias à criação da conta de acesso. Após a validação dos dados informados, o monitor passa a ter acesso às funcionalidades de gerenciamento de monitorias.

Figura 14 – Tela de Cadastro de Monitor

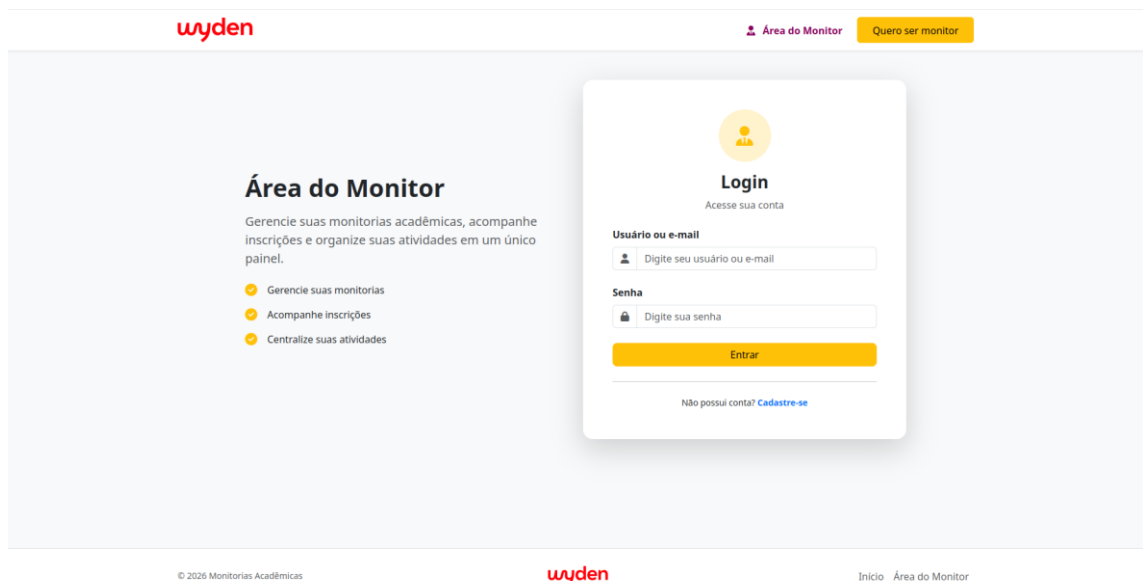
A imagem mostra a interface de usuário para o cadastro de um monitor na plataforma Wyden. No topo, há o logo "wyden" em vermelho à esquerda e, à direita, os links "Área do Monitor" e "Quero ser monitor" em um botão amarelo. O conteúdo principal é dividido em duas seções. À esquerda, sob o título "Seja um Monitor", há um texto explicativo: "Cadastre-se na plataforma para publicar monitorias, acompanhar alunos inscritos e desenvolver sua experiência acadêmica." Abaixo disso, há uma lista de benefícios com ícones de setas amarelas: "Publique monitorias", "Gerencie inscrições" e "Desenvolva experiência acadêmica". À direita, há um formulário branco com o título "Criar Conta" e o subtítulo "Cadastre-se como monitor". O formulário contém campos para "Nome de usuário" (com o exemplo "ex: nome.sobrenome"), "Email" (com o exemplo "seu@email.com"), "Senha" (com o ícone de olho para alternar visibilidade) e "Confirmar senha" (com o ícone de olho). Abaixo dos campos, há um botão amarelo "Criar Conta" e, no rodapé do formulário, o link "já possui conta? Entrar".

Fonte: Autores, 2026.

Tela de Login (login.html)

Responsável pela autenticação dos monitores cadastrados. A interface permite o acesso por meio de nome de usuário ou endereço de e-mail e senha. Após a validação das credenciais, o sistema concede acesso às áreas restritas de gerenciamento da plataforma.

Figura 15 – Tela de Login



Fonte: Autores, 2026.

Tela de Criação de Monitoria (criar_monitoria.html)

Destinada ao cadastro de novas monitorias pelos usuários autorizados. O formulário contempla os campos título, descrição, data de realização, categoria, benefício e quantidade de vagas disponíveis.

Figura 16 – Tela de Criação de Monitoria

Fonte: Autores, 2026.

Tela de Edição de Monitoria (editar_monitoria.html)

Permite a alteração das informações previamente cadastradas em uma monitoria. Os dados são apresentados de forma pré-preenchida, possibilitando sua atualização pelo monitor responsável.

Figura 17 – Tela de Edição de Monitoria

A interface de usuário para a edição de uma monitoria, intitulada "wyden". No topo direito, há uma saudação "Olá, aryane.andrade" e um botão "Sair". O formulário principal, "Editar Monitoria", contém os seguintes campos:

- Título da monitoria:** Um campo de texto com o valor "Do zero à nuvem: deploy na AWS".
- Descrição:** Um campo de texto com o valor "Configure servidores, banco de dados e domínio na AWS do jeito que o mercado faz."
- Categoria:** Um menu suspenso com o valor "TI".
- Benefício:** Um menu suspenso com o valor "CERTIFICADO".
- Data de Início:** Um campo de data com o valor "09 / 23 / 2026" e um ícone de calendário.
- Vagas disponíveis:** Um campo de texto com o valor "15" e um ícone de lupa.

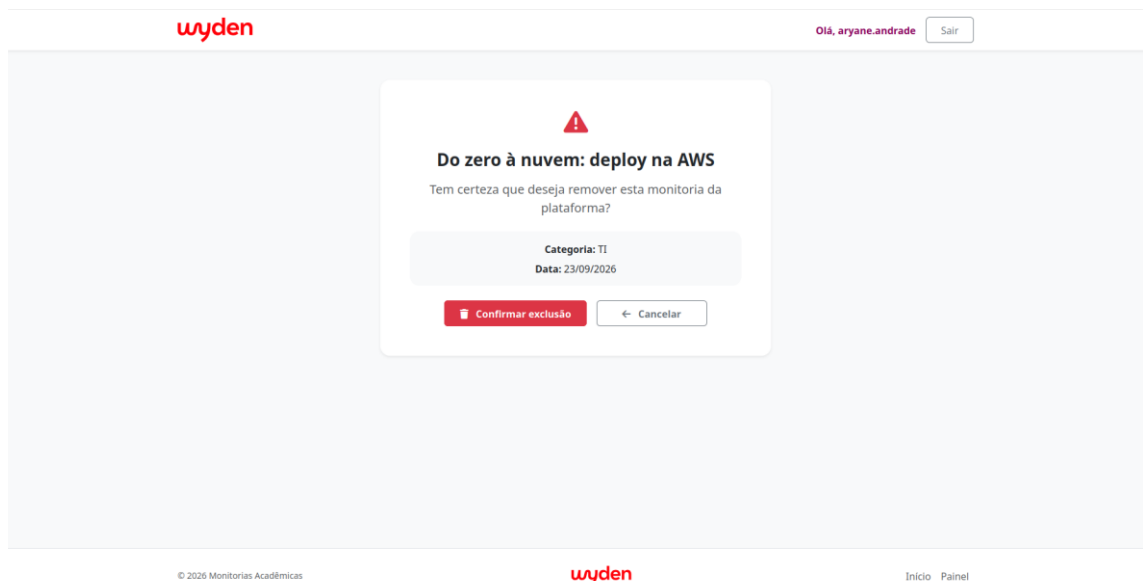
Na base do formulário, há três botões: "Salvar alterações" (amarelo), "Excluir" (vermelho) e "Cancelar" (cinza). No rodapé, há o copyright "© 2026 Monitorias Acadêmicas", o logo "wyden" e os links "Início" e "Painel".

Fonte: Autores, 2026.

Tela de Confirmação de Exclusão (excluir_monitoria.html)

Responsável por solicitar a confirmação da exclusão de uma monitoria cadastrada. A tela exibe as informações básicas do registro selecionado e disponibiliza a opção de confirmação da remoção.

Figura 18 – Tela de Confirmação de Exclusão de Monitoria

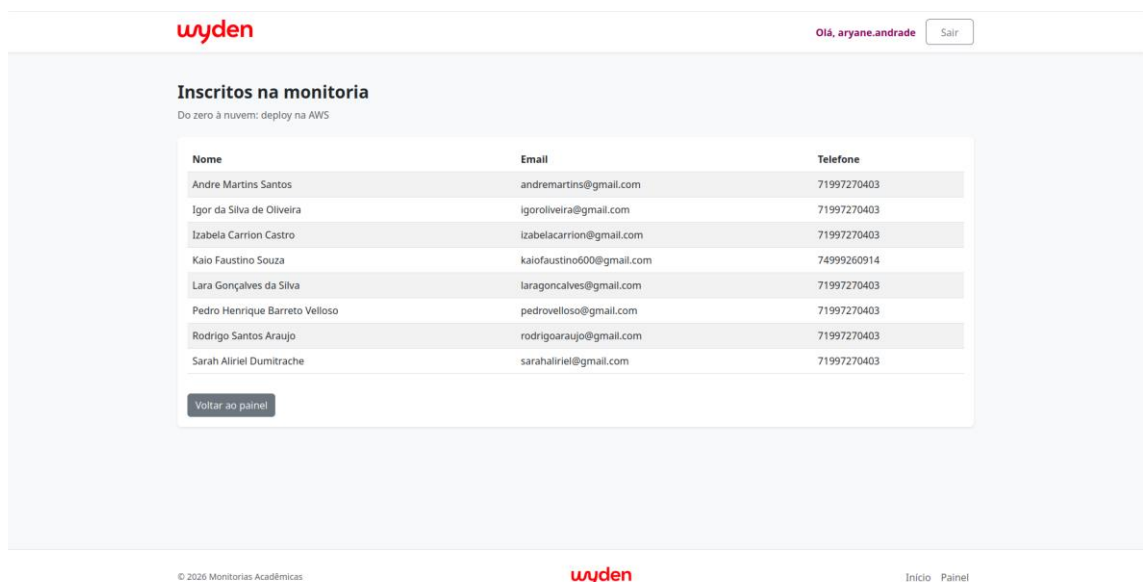


Fonte: Autores, 2026.

Tela de Lista de Inscritos (inscritos_monitoria.html)

Apresenta a relação dos alunos inscritos em determinada monitoria, exibindo nome, e-mail e telefone dos participantes. O acesso a essa funcionalidade é restrito ao monitor proprietário da monitoria.

Figura 19 – Tela de Lista de Inscritos

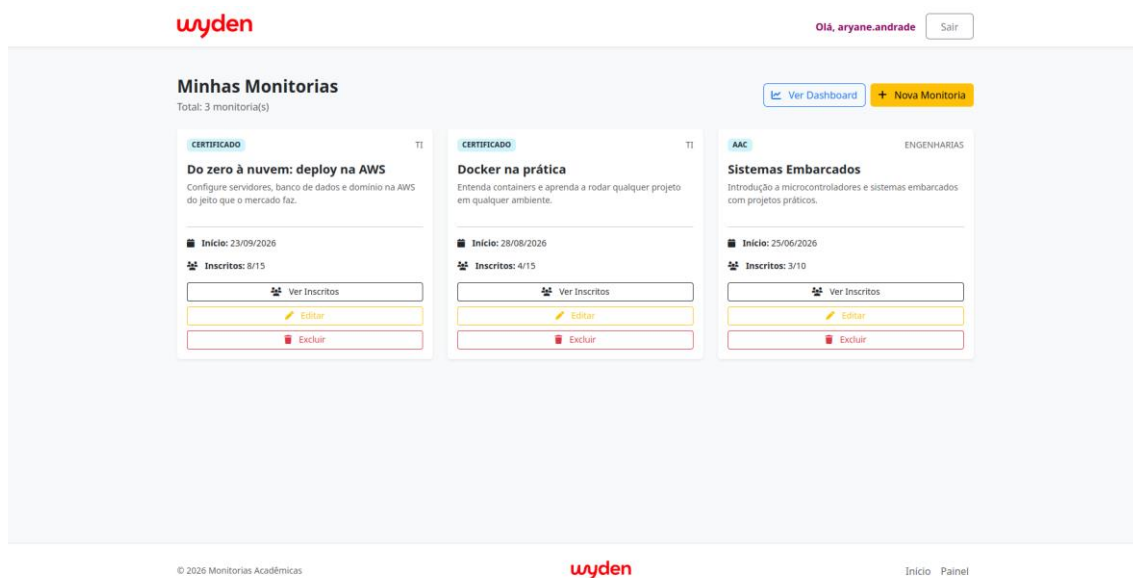


Fonte: Autores, 2026.

Tela do Painel do Monitor (painel_monitor.html)

Disponibiliza uma visão consolidada das monitorias cadastradas pelo usuário autenticado. Por meio dessa interface, o monitor pode acessar funcionalidades de edição, exclusão e consulta dos inscritos em suas monitorias.

Figura 20 – Tela do Painel do Monitor

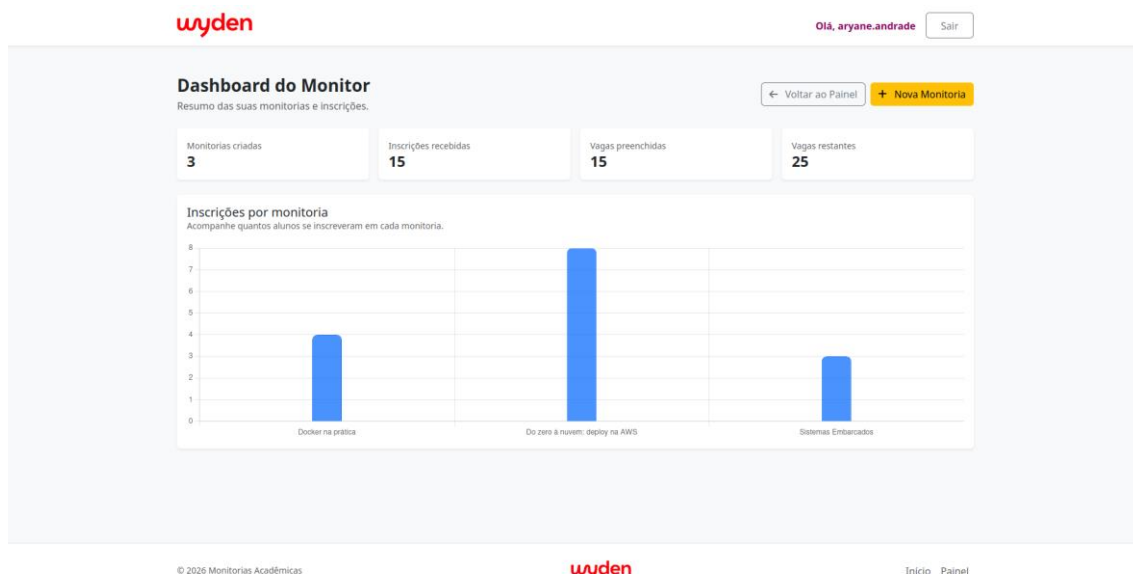


Fonte: Autores, 2026.

Tela do Dashboard (dashboard.html)

Apresenta indicadores gerenciais relacionados às monitorias cadastradas, incluindo métricas quantitativas de monitorias e inscrições, além de gráficos destinados ao acompanhamento da participação dos alunos em cada monitoria.

Figura 21 – Tela de Dashboard do Monitor



Fonte: Autores, 2026.

5.9 Telas de Diálogo

As interações de diálogo implementadas na plataforma incluem mensagens de feedback ao usuário exibidas após a execução de ações:

- Mensagem de sucesso na inscrição: confirmação exibida após o registro bem-sucedido de uma inscrição em monitoria.
- Mensagem de erro por monitoria lotada: alerta informando que a monitoria não possui vagas disponíveis.
- Mensagem de erro por e-mail duplicado: alerta informando que o e-mail informado já está cadastrado para a monitoria selecionada.
- Mensagem de sucesso no cadastro de monitoria: confirmação exibida após a criação bem-sucedida de uma nova monitoria.
- Tela de confirmação de exclusão: diálogo de confirmação exibido antes da remoção definitiva de uma monitoria.

5.10 Layout de Relatórios

A versão atual da Plataforma de Monitorias Acadêmicas não dispõe de funcionalidade de geração de relatórios estruturados em formatos exportáveis, como PDF ou CSV. Esta característica constitui uma limitação conhecida do sistema em sua versão atual,

identificada na etapa de consultoria do negócio. A implementação de relatórios de inscrições por monitoria, relatórios consolidados por período e exportação de dados em formato CSV é prevista como melhoria para versões futuras da plataforma.

5.11 Especificação de Programas – Módulos do Sistema

O projeto está organizado em três módulos Django principais, cada um com responsabilidade bem definida:

- Módulo contas: responsável pelos processos de autenticação e gerenciamento de usuários. Contém o backend de autenticação personalizado EmailOrUsernameBackend, o formulário RegisterForm para cadastro de novos usuários, o formulário LoginForm para autenticação, e as views register_view, login_view e logout_view.
- Módulo monitorias: módulo central do domínio da aplicação, contendo os modelos Monitoria, Inscricao, Categoria e Beneficio; os formulários MonitoriaModelForm e InscricaoModelForm; e todas as views relacionadas ao ciclo de vida das monitorias e inscrições.
- Módulo app: módulo de configuração da aplicação Django, responsável pela definição das configurações globais (settings.py), pelo roteamento principal de URLs (urls.py) e pelos templates globais da aplicação.

6. IMPLEMENTAÇÃO

6.1 Desenvolvimento das Aplicações

O desenvolvimento da Plataforma de Monitorias Acadêmicas seguiu o processo de desenvolvimento de software estruturado pela disciplina de Engenharia de Software, compreendendo as etapas de análise de requisitos, projeto de arquitetura, implementação, testes e implantação. A metodologia adotada foi incremental, com o desenvolvimento progressivo das funcionalidades a partir das mais essenciais (listagem pública e inscrição) até as mais avançadas (painel de monitor e dashboard).

Organização do Código e Estrutura dos Módulos

A organização do código-fonte segue a estrutura convencional de projetos Django, com separação clara de responsabilidades entre módulos. O diretório raiz do projeto contém os arquivos de configuração de contêineres (docker-compose.yml, docker-compose.prod.yml, Dockerfile, entrypoint.sh) e o módulo app com as configurações globais. Os módulos de domínio contas e monitorias são organizados cada um com seus próprios models.py, views.py, forms.py, urls.py e diretório de templates.

Arquitetura MVT e Decisões Arquiteturais

O padrão arquitetural MTV (Model-Template-View) do Django foi aplicado de forma consistente em toda a implementação. Os Models encapsulam a lógica de domínio e as propriedades computadas das entidades. As Views implementam a lógica de controle, incluindo validações de autorização e processamento de formulários. Os Templates implementam a camada de apresentação, recebendo o contexto de dados das Views e renderizando o HTML final.

As decisões arquiteturais mais relevantes do projeto incluem: a implementação do backend de autenticação personalizado EmailOrUsernameBackend, que estende o sistema de autenticação padrão do Django para suportar login tanto por username quanto por endereço de e-mail; a restrição de acesso às views de edição e exclusão de monitorias por meio de querysets filtrados pelo usuário autenticado, garantindo que monitores não possam alterar monitorias de outros usuários; e a otimização das consultas na MonitoriaListView por meio dos métodos select_related() e annotate() do ORM do Django, reduzindo o número de consultas ao banco de dados.

6.2 Testes QA

A estratégia de qualidade de software adotada no projeto compreende a execução da suíte de testes automatizados do Django, complementada por testes manuais dos fluxos críticos da aplicação. A abordagem de testes é orientada pelos conceitos de verificação (o sistema foi construído corretamente?) e validação (o sistema correto foi construído?) da Engenharia de Software.

Estratégia de Testes e Verificação

A suíte de testes automatizados é executada por meio do comando `docker compose exec web python manage.py test`, que invoca o framework de testes do Django. Os testes cobrem as validações de domínio dos modelos *Monitoria* e *Inscricao*, os comportamentos das views de listagem pública e criação de monitorias, e os fluxos de registro e login de usuário. A análise de cobertura dos testes identificou que os módulos de dashboard e painel do monitor não possuem cobertura de testes automatizados na versão atual, constituindo uma lacuna a ser endereçada em trabalhos futuros.

Qualidade do Software e Conformidade com Requisitos

A qualidade da plataforma foi avaliada sob os atributos de qualidade definidos pela norma ISO/IEC 25010: funcionalidade (aderência aos requisitos funcionais especificados), confiabilidade (comportamento consistente em cenários de uso normal), usabilidade (interface intuitiva para os perfis de usuário identificados), manutenibilidade (código organizado e documentado), e portabilidade (execução consistente em diferentes ambientes por meio do Docker). A conformidade com os requisitos funcionais foi verificada por meio de testes de aceitação manuais realizados ao longo do desenvolvimento.

7. IMPLANTAÇÃO

7.1 Instalação

O processo de instalação da plataforma em ambiente de produção compreende a configuração do servidor de hospedagem, a instalação das dependências de infraestrutura e a inicialização dos serviços da aplicação. Os pré-requisitos de instalação são: servidor Linux (Ubuntu Server 22.04 LTS ou superior), Docker Engine (versão 24 ou superior) e Docker Compose (versão 2 ou superior).

O procedimento de instalação segue os seguintes passos: clonagem do repositório do projeto com o comando `git clone`; configuração das variáveis de ambiente a partir do arquivo `.env.example-prod`; e inicialização dos containers com o comando `docker compose -f docker-compose.prod.yml up -d --build`. O `entrypoint.sh` executa automaticamente as migrações de banco de dados e a coleta de arquivos estáticos durante a inicialização.

7.2 Migração e Conversão de Dados

As migrações de schema do banco de dados são gerenciadas pelo mecanismo de migrações do Django, que mantém um histórico de alterações na estrutura do banco de dados e permite a sua aplicação de forma controlada. O comando `python manage.py migrate`, executado automaticamente pelo `entrypoint.sh` na inicialização, aplica todas as migrações pendentes. Não foram necessários scripts de conversão de dados provenientes de sistemas legados, uma vez que a plataforma foi desenvolvida como uma solução nova, sem migração de dados históricos de planilhas.

7.3 Treinamento

O treinamento dos usuários da plataforma foi realizado de forma prática, por meio de demonstrações dos fluxos principais de utilização. Monitores foram orientados quanto ao processo de cadastro, criação de monitorias e acompanhamento de inscrições. Alunos foram apresentados à interface pública de listagem e inscrição em monitorias. A equipe técnica responsável pela operação foi treinada nos procedimentos de manutenção, backup e atualização da plataforma.

7.4 Validação

A validação do sistema em ambiente de produção compreendeu a verificação dos seguintes critérios: acessibilidade da interface pública de listagem de monitorias pela URL de produção; funcionamento correto do fluxo de inscrição de alunos, incluindo as validações de disponibilidade de vagas e unicidade de e-mail; acessibilidade e funcionamento das funcionalidades restritas a monitores autenticados; disponibilidade do painel e do dashboard; e funcionamento correto do certificado TLS, com redirecionamento automático de HTTP para HTTPS.

8. PRODUÇÃO

8.1 Homologação e Funcionamento do Sistema

A Plataforma de Monitorias Acadêmicas encontra-se em operação em ambiente de produção na infraestrutura de nuvem Amazon Web Services, acessível pelo endereço `monitorias.andops.com.br`. A implantação em produção utiliza o arquivo `docker-compose.prod.yml`, que define a configuração dos serviços web (Gunicorn + Django), nginx (proxy reverso e servidor de estáticos) e, em ambiente de produção, pressupõe a utilização de um banco de dados externo ao servidor de aplicação.

A configuração de segurança do ambiente de produção inclui as seguintes medidas: habilitação de HTTPS com redirecionamento automático de HTTP (`DJANGO_SECURE_SSL_REDIRECT=True`); configuração de cookies de sessão e CSRF com atributo Secure (`DJANGO_SESSION_COOKIE_SECURE` e `DJANGO_CSRF_COOKIE_SECURE`); e definição de origens confiáveis para proteção CSRF (`DJANGO_CSRF_TRUSTED_ORIGINS`). O certificado TLS é obtido e renovado automaticamente pelo Certbot em integração com o Let's Encrypt, no modo `--webroot`.

A operação do sistema em produção confirma a viabilidade da arquitetura adotada e a adequação das tecnologias selecionadas às necessidades do projeto. O fluxo de requisições segue a cadeia Cliente → Nginx (porta 443, HTTPS) → Gunicorn (porta 8000, HTTP interno) → Django (processamento) → PostgreSQL (persistência), conforme projetado na arquitetura da aplicação.

Os benefícios obtidos com a implantação da plataforma incluem: centralização de todas as informações de monitorias em um único ponto de acesso; eliminação do processo manual de controle de vagas e inscrições; disponibilização de uma interface pública acessível a todos os alunos da instituição; e disponibilização de um dashboard de métricas que permite aos monitores e à coordenação acadêmica acompanhar o desempenho do programa de monitorias.

CONCLUSÃO

O presente trabalho documentou o desenvolvimento da Plataforma de Monitorias Acadêmicas, uma solução web desenvolvida para centralizar e modernizar o processo de gestão de monitorias do Centro Universitário UniRuy Wyden. O projeto foi concebido a partir da identificação de um problema real na instituição — a descentralização e a informalidade do processo vigente de divulgação e controle de monitorias — e desenvolvido com rigorosa aplicação dos princípios e metodologias da Engenharia de Software, compreendendo as etapas de levantamento de requisitos, modelagem de dados, especificação de regras de negócio, implementação, testes e implantação.

Os objetivos definidos para o sistema foram integralmente alcançados. A plataforma disponibiliza todas as funcionalidades especificadas nos requisitos funcionais: listagem pública de monitorias com busca e filtros, inscrição de alunos com controle automático de vagas, painel de gestão para monitores, dashboard com métricas de desempenho e sistema completo de autenticação com suporte a login por e-mail ou username. A solução elimina a dependência de planilhas eletrônicas e grupos de mensagens, substituindo o processo informal e suscetível a erros por um sistema organizado, centralizado e acessível a toda a comunidade acadêmica.

A aplicação dos conceitos da Engenharia de Software ao longo do desenvolvimento do projeto revelou-se fundamental para a qualidade e a coerência da solução entregue. O levantamento sistemático de requisitos permitiu a identificação clara das necessidades dos usuários e das restrições do sistema. A modelagem de dados conceitual e física assegurou a integridade e a consistência das informações armazenadas. A adoção do padrão arquitetural MTV do Django, combinada com a contêinerização por Docker e a implantação em nuvem AWS, resultou em uma solução portátil, manutenível e escalável. A definição de um plano de testes e a implementação de uma suíte de testes automatizados contribuíram para a verificação da conformidade do sistema com os requisitos especificados.

Como perspectivas para trabalhos futuros, identificam-se as seguintes possibilidades de evolução da plataforma: implementação de sistema de notificações por e-mail para confirmação de inscrições e alertas de disponibilidade; desenvolvimento de funcionalidade de geração de relatórios exportáveis em formatos PDF e CSV; criação de

interface CRUD dedicada para gerenciamento de categorias e benefícios, eliminando a dependência da interface administrativa do Django; implementação de suporte a fila de espera e cancelamento de inscrições; e estabelecimento de um pipeline de integração e entrega contínua (CI/CD) para automatizar o processo de testes e implantação. A concretização dessas melhorias permitirá a evolução da plataforma para um sistema ainda mais completo e aderente às necessidades da gestão acadêmica institucional.

REFERÊNCIAS

AMAZON WEB SERVICES. Amazon EC2: virtual servers in the cloud. Disponível em: <https://aws.amazon.com/ec2/>. Acesso em: jun. 2026.

BECK, Kent et al. Manifesto para Desenvolvimento Ágil de Software. 2001. Disponível em: <https://agilemanifesto.org/iso/ptbr/manifesto.html>. Acesso em: jun. 2026.

BOEHM, Barry W. Software Engineering Economics. Englewood Cliffs: Prentice-Hall, 1981.

DJANGO SOFTWARE FOUNDATION. Django documentation. Versão 5.x. 2024. Disponível em: <https://docs.djangoproject.com/>. Acesso em: jun. 2026.

DOCKER INC. Docker documentation. 2024. Disponível em: <https://docs.docker.com/>. Acesso em: jun. 2026.

ISO/IEC. ISO/IEC 25010:2011: Systems and software engineering – Systems and software Quality Requirements and Evaluation (SQuaRE) – System and software quality models. Geneva: ISO, 2011.

NGINX INC. NGINX documentation. 2024. Disponível em: <https://nginx.org/en/docs/>. Acesso em: jun. 2026.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. PostgreSQL 16 documentation. 2024. Disponível em: <https://www.postgresql.org/docs/>. Acesso em: jun. 2026.

PRESSMAN, Roger S.; MAXIM, Bruce R. Engenharia de Software: uma abordagem profissional. 8. ed. Porto Alegre: AMGH, 2016.

SOMMERVILLE, Ian. Engenharia de Software. 10. ed. São Paulo: Pearson Education do Brasil, 2018.

WOHLIN, Claes et al. Experimentation in Software Engineering. Berlin: Springer, 2012.